COMP0214 Coursework
Introduction to Machine Learning
University College London

Aayan Islam – 24102520

December 2025

1 Analyzing Retail Fuel Price Dynamics in New South Wales (2016–2025)

1.1 Data Cleaning and Visualization

1.1.1 Cleaning & Summary

fuelPrice_NSW.csv contains fuel prices of various fuel stations from the 1^{st} August 2016 to the 31^{st} August 2025. When reading the csv file, the dimensions are found to be 98926×6. One change that had to made was converting the PriceUpdateDate into date/time format. This was to ensure that there were no issues when handling dates. Additionally, the values in price were made numeric using **pd.to_numeric** to ensure that they were treated as integers/floats.

	ServiceStationName	FuelCode	PriceUpdatedDate	Price	Latitude	Longitude
0	7-Eleven Minchinbury	E10	2025-08-31 22:10:00	159.9	-33.778213	150.808089
1	7-Eleven Minchinbury	U91	2025-08-31 22:10:00	163.9	-33.778213	150.808089
2	7-Eleven Minchinbury	P95	2025-08-31 22:10:00	178.9	-33.778213	150.808089
3	7-Eleven Minchinbury	P98	2025-08-31 22:10:00	185.9	-33.778213	150.808089
4	7-Eleven Blacktown	P98	2025-08-31 18:44:00	188.9	-33.754838	150.891467

(a) Raw Data (Before)

	ServiceStationName	FuelCode	PriceUpdatedDate	Price
0	7-Eleven Blacktown	E10	2016-08-01	101.9
1	7-Eleven Blacktown	E10	2016-08-02	99.9
2	7-Eleven Blacktown	E10	2016-08-03	99.9
3	7-Eleven Blacktown	E10	2016-08-04	98.9
4	7-Eleven Blacktown	E10	2016-08-06	99.9

(b) Cleaned Data (After)

Figure 1: Comparison of the dataset structure before and after cleaning.

To remove duplicates, the pandas function **drop_duplicates()** was used which removed 2905 entries. Additionally, to check for implausible values, the general rule of $Q_1 - 1.5 \times IQR$ and $Q_3 + 1.5 \times IQR$ was used. This was to ensure there are no unreasonable values which can affect the predictions later on. This outlier detection removed 380 entries bringing the total entries down from 98926 to 95640.

Finally, the data was aggregated(joined) for each FuelCode and data. This was achieved by using **df.groupby(['StationName','FuelCode','PriceUpdateDate'])['Price'].min(),reset_index()** which combined entries with the same station name, fuel code and date and used the smallest value out of all those entries. **reset_index()** reorganizes the entries so that they are much easier to read in the table

The following table showcases the csv data before and after cleaning:

Table 1: Dataset Summary Before and After Cleaning

Metric	**Raw Data**	**Cleaned Data**
Total Records (Size)	98,925	38,856
Date Range Start	2016-08-01	2016-08-01
Date Range End	2025-08-31	2025-08-31
Fuels Sold	E10, P95, P98, PDL, U91, LPG, DL	

1.1.2 Visualize

The task is to produce some plots for one fuel station. The fuel station of choice was **7-Eleven Blacktown**. This was gathered by using a boolean mask on the station data so that only the specific station values were kept.

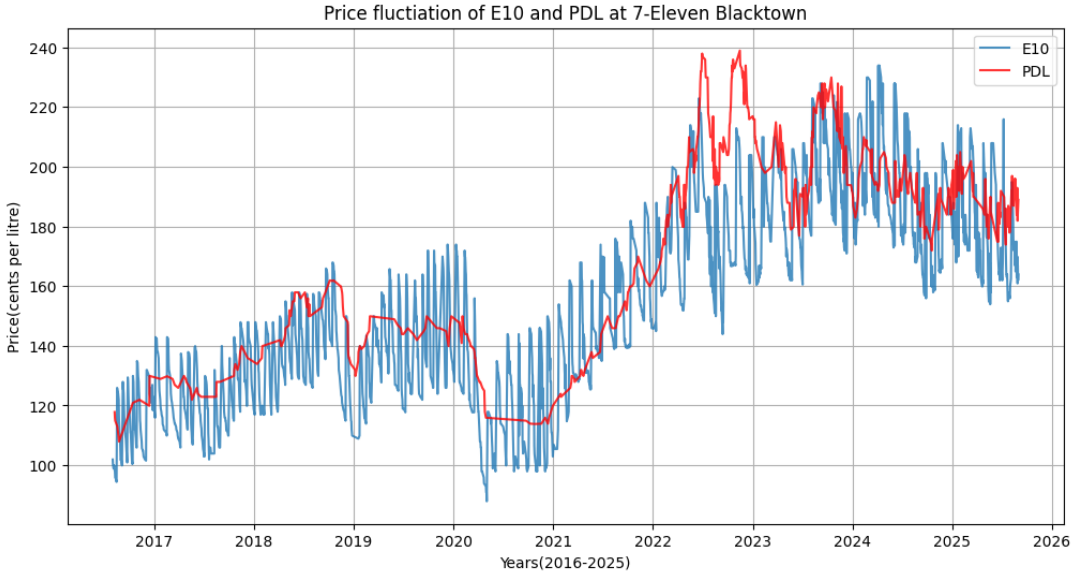

Figure 2: Time series plot

Based off the figure, it can be seen that there is a general increase of fuel price as time increases. This is most likely due to increased customer demand as well as higher production costs. Additionally, fuel prices dropped by a large margin between 2020 and 2021, most probably due to the COVID-19 pandemic which lead to world-wide lock downs reducing fuel usage/demand.

Additionally, a box plot was made to compare daily prices across all available fuel codes at **7-Eleven Blacktown**.

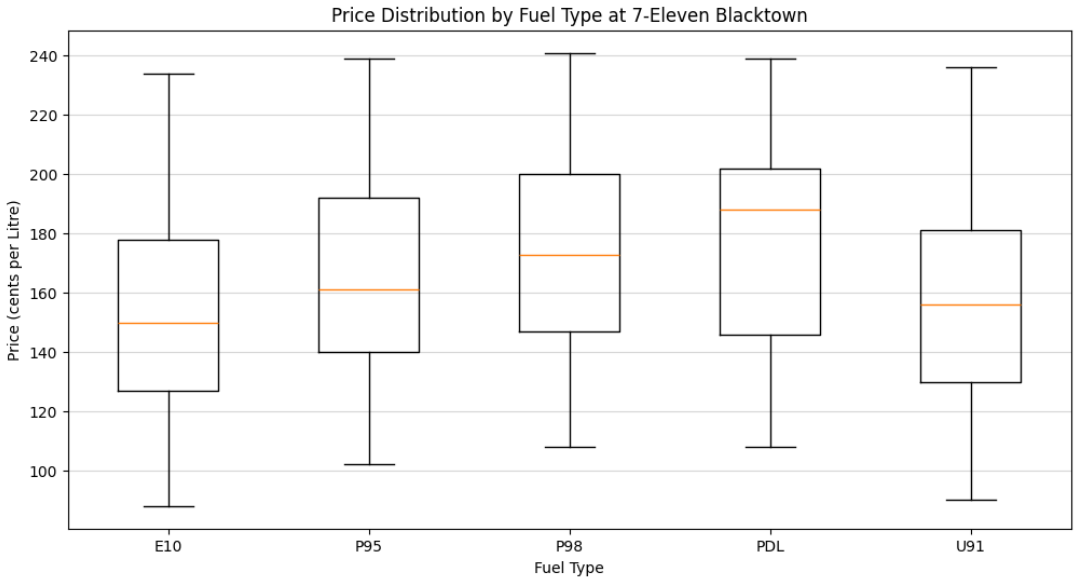

Figure 3: Box plots of all fuels available at 7-Eleven Blacktown

The box-plots show a clear pricing hierarchy ($E10 < U91 < P95 < P98 < PDL$) based on median costs, yet reveal that all fuel types converge to a nearly identical maximum peak of roughly 240 cents per liter. This means that while baseline costs differ, extreme market shocks affect all fuel types equally.

1.2 Forecasting Next-Day Prices

1.2.1 Problem setup & data split

To be able to predict/forecast a new price for the next day, the previous seven days are used as inputs so that the algorithm can capture weekly patterns. The target is the predicted price of the next day. Our unit of time are days as there is a price value for each day

A suitable split of data would be splitting the entire set into a training, validation and testing set with a 60/20/20 share. This would mean that training is from 2016 – Dec 31 2022, validation is from Jan 1 2023 – Dec 31 2023 and the testing set is from Jan 1 2024 – Aug 2025. By splitting the data using a chronological order, we can train the model so that it uses past data to predict future values.

To prevent data leakage, the split had to be purely chronological. There could not be any random selection as that may lead to recent dates being used to train the model whereas we want historical data to be used in training. Since the training was done by using the last seven days, there had to be data cutoffs at specific points. This was so that the first dates in the validation set did not end up in the training set as well as from the test set into the validation set.

1.2.2 Model design & training

One such feature implemented were the lag features which contained the prices of the previous seven days. This was done to record weekly fluctuations. This was done instead of a single day to allow a larger amount of historical data to help the model learn.

Additionally, a different number of lag days (10,12,14,21,50) were also tested to see what would perform better. It was noticed that increasing the numbers to a certain extent would decrease the validation MSE. However, it was also observed that this would increase the test MSE suggesting that the data was being overfit. Therefore it was decided that 7 was the optimal number of lags.

A variety of models were tested on the data from **7-Eleven Blacktown E10** and evaluated to see which one would be the most effective.

- **Linear Regression:**
  - Validation set MSE: 127.84
  - Test set MSE: 149.75
- **Random Forest:**
  - n_estimators = 120, max_depth = 10, random_state = 42
  - Validation set MSE: 128.15
  - Test set MSE: 142.35
- **Gradient Boosting:**
  - n_estimators=120, learning_rate=0.1, max_depth=5, random_state=42
  - Validation set MSE: 135.72
  - Test set MSE: 151.04
- **Support Vector Machines(SVMs):**
  - C=100, epsilon=0.1
  - Validation set MSE: 137.50
  - Test set MSE: 160.37

Although linear regression and random forest performed very similarly, random forest had an average MSE of 135 compared to Linear regression's average MSE of 138. Therefore, random forest was used for the final predictions

The procedure to train the model involved only using the relevant station and fuelcode prices. The data was then sorted chronologically and each lag feature was computed. As some days would cause leakages, they were dropped. Using the new data sets, new training, validation and testing sets were created. The random forest model was then trained using **.fit** which automatically carried out gradient descent to work out the optimal values

1.2.3 Evaluation & visualization

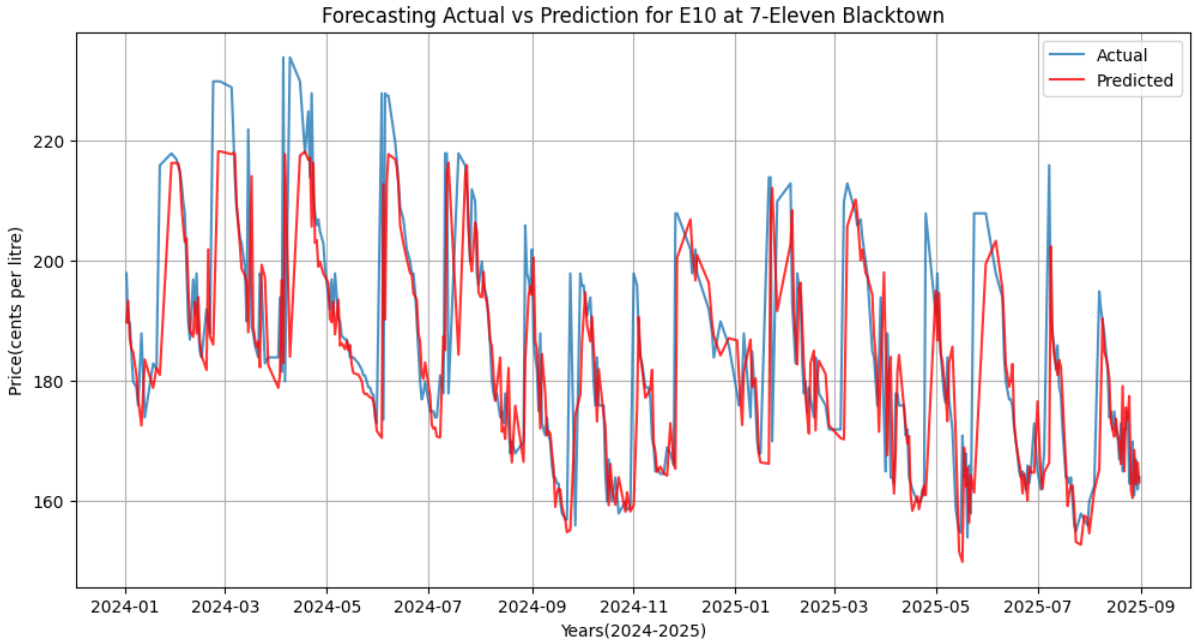

Figure 4: Predicted and Actual prices of E10

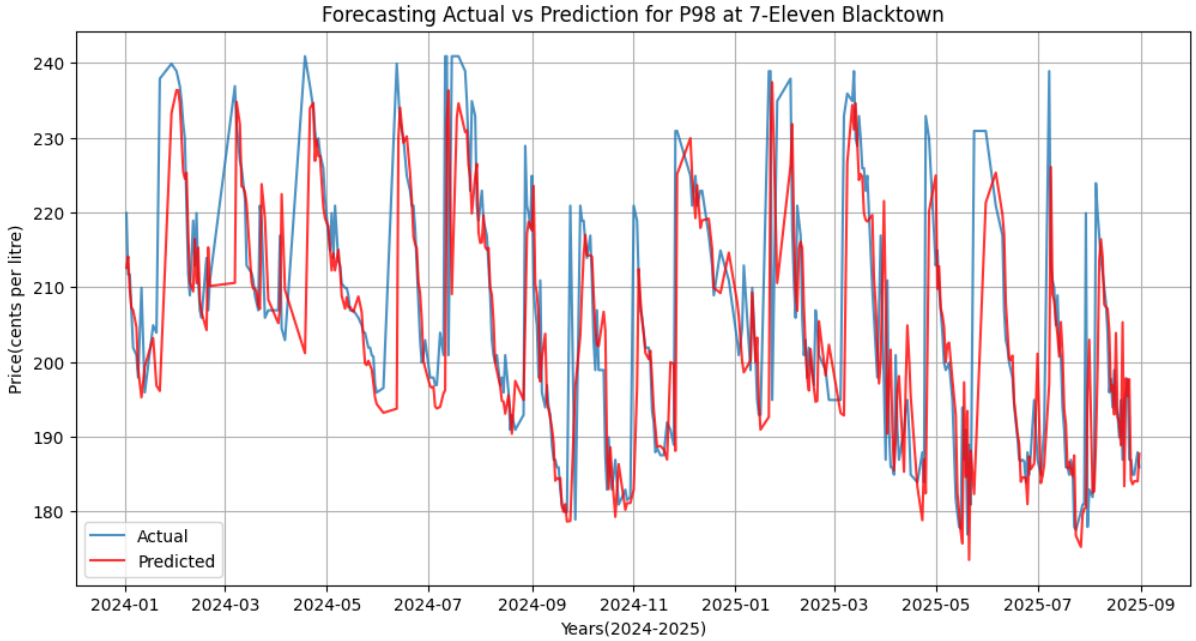

Figure 5: Predicted and Actual prices of P98

```
Processing fuel code E10 for station 7-Eleven Blacktown
Validation Check | Validation MSE: 128.15 | Validation RMSE: 11.32 | stnd dev: 31.69
Model Trained | Test MSE: 142.35 | Test RMSE: 11.93 | stnd dev: 31.69
Last Date in Data: 2025-08-31 with price: 163.90
Forecast for Next Day: 163.10 cents
```

Figure 6: Output values

This model achieves an MSE of $142.35 and RMSE of $11.39. Given that the mean and standard deviation are $153 and $31.69 respectively, this model has effectively learned the patterns in the data.

The model can follow the data very well. However, it can be seen that the model struggles to follow at the figures turning points. This is to the previous (t-1) day having a greater weighting. Therefore, there is a greater error at turning points where the model struggles to anticipate these sharp fluctuations in price until it has already occurred.

1.3 Applications & Limitations

Practical Use: The model can be integrated into a mobile application to allow for customers to view daily fuel prices and predictions. As the customer must purchase fuel everyday, it is more suitable that customers are recommended a volume of fuel. If the model predicts a price increase for the next day, the application would recommend the customer to fill up their tank today. Conversely, if the model predicts a price drop for the next day, the application should suggest to only purchase the minimum amount of fuel needed for the rest of the day. They can then purchase more fuel the next day at a cheaper price. These predictions can allow customers to minimize their average cost on fuel per day

Limitation & remedy: A limitation of this current model is that it relies on historical price lags. This causes an issue with the predictions as it struggles to anticipate the sudden price changes for the fuels until it has already occurred. This is most likely due to how a higher weighting is given to more recent days in the lag.

To address this problem, other features such as global oil prices could be implemented. This is because global oil prices tend to rise a few days before stations modify their prices. This can act as a leading indicator which can help predict sharp changes in fuel prices. To measure this, The model would need to be retrained with this new feature to compare the MSE and RMSE scores against the current model. The main part to look at would be where the sharp fluctuations occur and check whether the new model is correctly tracking throughout all the testing data.

2 WiFi Signal for Indoor Localization

2.1 Multi-Linear Regression Model for Indoor Localization

Based off **Wifi_train_dataset.csv**, all the RSSI and MAC values are within a single cell separated by commas. Therefore, the data needs to be broken up using the commas which assigns the RSSI to its respective MAC address. Each MAC address represents a beacon. Not every input will include very beacon. This process revealed that there are 562 unique routers.

Table 2: Dataset dimensions before and after feature extraction.

Dataset	Initial Dimensions	Final Dimensions
Training Set	$(2880, 4)$	$(2880, 562)$
Testing Set	$(720, 4)$	$(720, 562)$

Since some of the inputs in the training set didn't have all the routers, they can be considered as NaN values. To remedy this, for any router that did not have a signal, it was given a value of -110. The reason for this value is because in decibels (dB) a smaller value represents a weaker signal. Using -110, the model understands that these beacons are giving very weak signals. Through trial and error, -110 managed to produce the lowest MSE compared to -95,-100, and -150.

To carry out linear regression, the training data was split into a training set and a validation set. This is so that we can check our results in the validation set to generate the MSE values . This was done through an 80/20 split. After carrying out regression, the model had a mean squared error (MSE) of **23.09**. This means that on average, the error would be around 4.5 meters. This provides a baseline to our models. A reason our MSE for the model is so high is most likely due to the data being non-linear. This means that linear regression cannot capture all the features which creates a high MSE.

2.2 Neural Network Model for Indoor Localization

To further improve our model, a neural network was used instead. This makes use of neurons and multiple activation functions to predict values.

To minimize the MSE as much as possible, multiple models were trialed and tested:

- A Linear neural network with stochastic gradient descent.
  562 input neurons → 2 output neurons
  MSE: 22.05
- A neural network with 2 hidden layers using stochastic gradient descent.
  562 input neurons → 128 → 64 → 2 output neurons
  MSE: 15.91
- A neural network with 2 hidden layers using Adam (Adaptive moment estimation).
  562 input neurons → 128 → 64 → 2 output neurons
  MSE: 12.43
- A neural network with 3 hidden layers using Adam.
  562 input neurons → 512 → 128 → 64 → 2 output neurons
  MSE: 8.39

Based off these different models, the 3 hidden layer neural network performed the best. Therefore this model would be used to predict the coordinates for the test values. Comparatively, the nn model improves on predicting the data by **63.6%**. This occurs due to the non-linearity of the signals. These signals most likely decay following an inverse square law, therefore a linear model would struggle to fit a line for all the points. The neural network can make use of non-linear functions such as ReLU which can allow for non-linear mapping. Additionally, the 3 hidden layer network generated a mean percentage error (MPE) of **7.91%**.

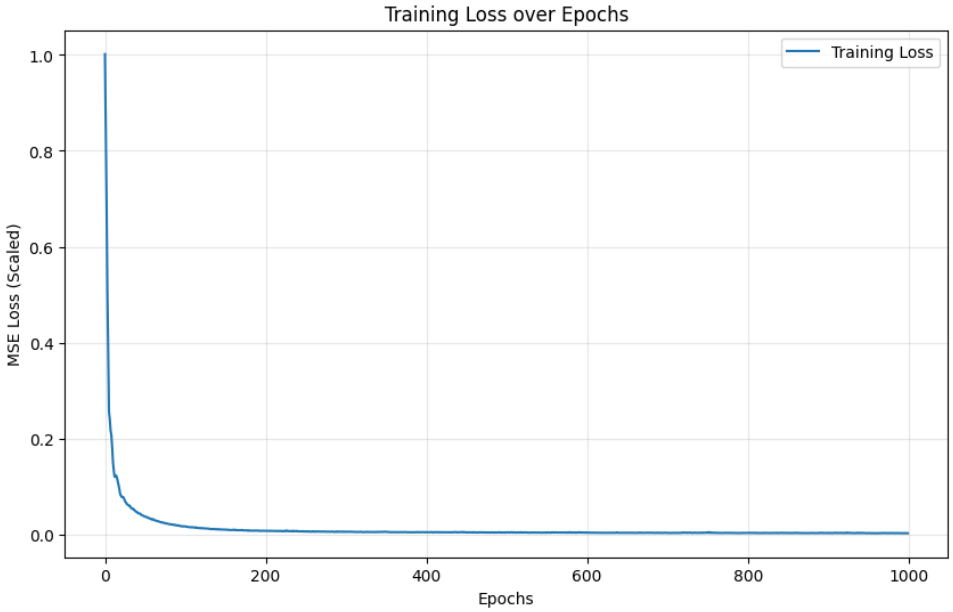

Figure 7: Training loss against epochs

2.3 Dimensionality Reduction for Feature Visualization

From the neural network, the 128 neuron layer was used. The justification for this was that this shows the networks internal representation of the surrounding environment before it is compressed into cartesian coordinates (X,Y). Using this layer, we can turn it into a tensor which is used with the K-mean clustering algorithm. To determine the ideal number of clusters, we can run tests with varying numbers of clusters and look for the "elbow" of the graph which signifies the best cluster value.

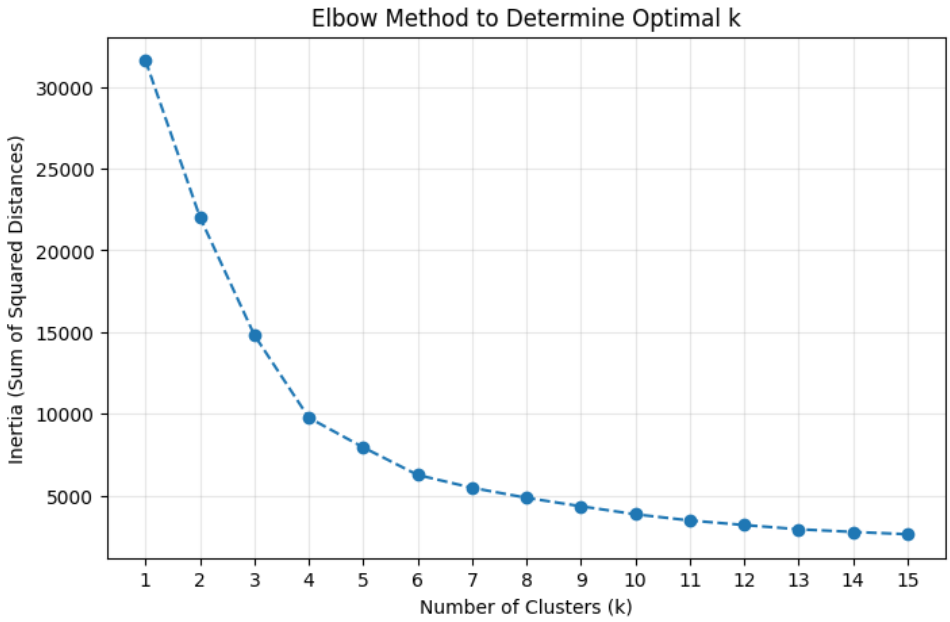

Figure 8: Varying k values to find the ideal number of clusters

Based off figure 8, the ideal number lies between 4,5 or 6. Based off testing and visualization, using k=6 proved to give the best clusters.

Whilst the data has been clustered through K-means, it still has 128 features. Humans can only perceive information up to the third dimension. Therefore, a dimensional reduction algorithm needs to be used to allow the data to be visualized and presentable. The two main algorithms used are principle component analysis (PCA) and t-distributed stochastic neighbor embedding (t-SNE).

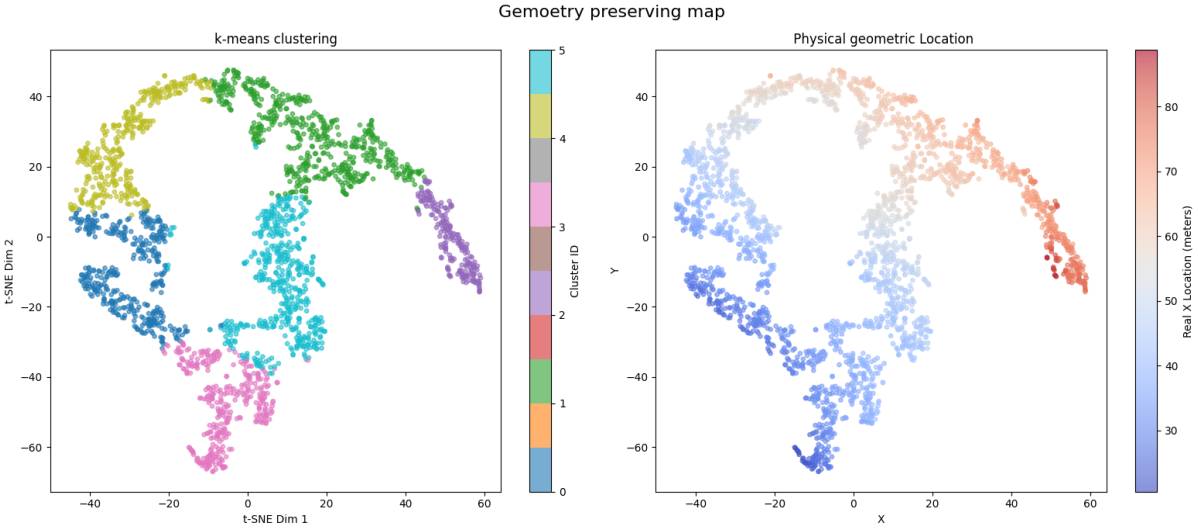

Figure 9: t-SNE reduction with physical geometry

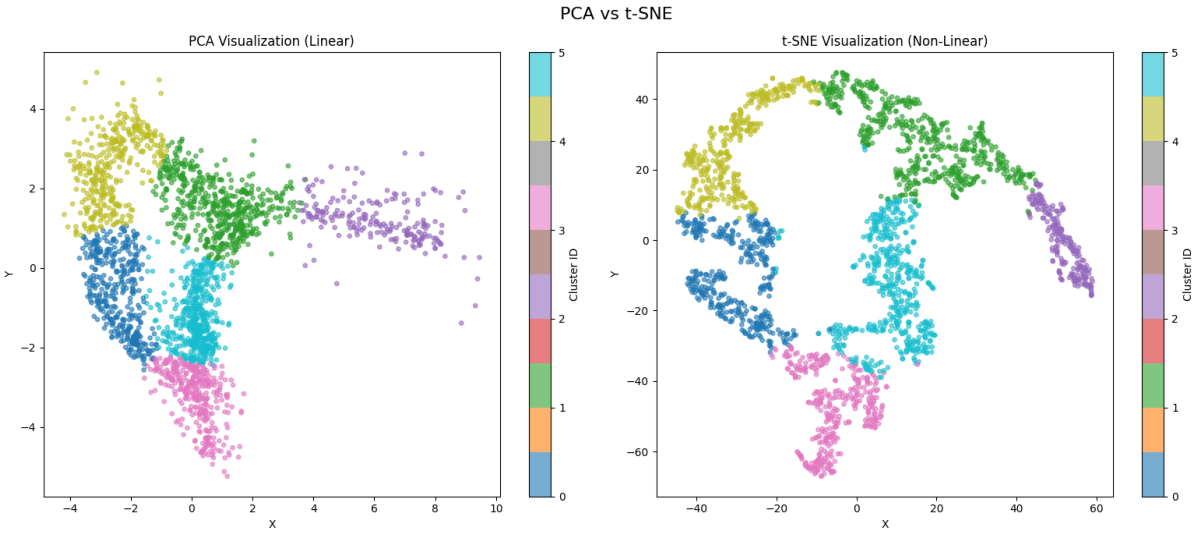

Figure 10: t-SNE vs PCA for dimensional reduction

When comparing t-SNE with PCA, a visible difference is that PCA keeps the data much closer together whereas t-SNE is able to spread them out. This is because PCA aims to reduce the data by capturing as much variance possible through a linear transformation. This means that it struggles with capturing the variance on non-linear data. This is clear in figure 10 where the beacon points are much more crammed together using PCA.

t-SNE focuses on preserving topology, whereas PCA focuses on preserving information. Crucially, the right subplot in figure 9 shows a smooth color gradient. The points corresponding to the left of the building are topologically distant from the points on the right. As seen in the right subplot of figure 9, points fade smoothly from blue (Low X) to red (High X) This confirms the network has learned a geometry-preserving map of the indoor space.

2.4 Prediction and Analysis on Test Data

To predict values of the test set, the existing model that was previously trained must be re-used. Using the existing scales, the test data is normalized and then passed into the model. Through the **inverse_-transform** function, we can return the predicted values back to their original scale. These values are then stored in a txt file called **zcabais.txt**

X	Y
54.560692	-30.919601
70.691116	-42.651657
37.346561	-19.467146
45.349277	-15.257744
31.933271	-21.128298
34.500793	-30.634739
48.735039	-33.195778
75.763840	-41.759144
36.749443	-32.611706
30.620583	-29.366474

Table 3: First 10 predicted coordinates of test set

Github Repository

https://github.com/iaayan3913/COMP214_Machine_Learning_Coursework